

Examen Pratique – Développement et Déploiement d’une Solution Web Full Stack

Filière : Développement Digital – Option Web Full Stack

Durée : 4h00

Barème : /60 pts

Contexte

La société **BiblioTech Cloud** souhaite mettre en place une solution web complète permettant la gestion des livres, des membres et des emprunts.

La solution demandée comprend :

- une **application Laravel** assurant :
 - la gestion de la base de données MySQL
 - une interface web avec **vues Blade**
 - une **API REST**
- une **application React** utilisant **Redux** pour la gestion de l’état global
- une base de données **MySQL hébergée sur AlwaysData**
- un déploiement en ligne :
 - **application Laravel** sur **Render** ou **Vercel**
 - **application React** sur **Vercel**

L’objectif est d’obtenir une solution accessible en ligne, composée d’une application Laravel web + API et d’une application React, avec communication entre elles via HTTP.

Dossier 1 : Base de données distante sur AlwaysData (10 pts)

Travail demandé

1. Créer une base de données MySQL nommée **bibliotech** sur AlwaysData. **(2 pts)**
2. Créer les tables suivantes avec des champs adaptés au contexte de gestion d’une bibliothèque : **(4 pts)**
 - **livres**
 - **membres**
 - **emprunts**
3. Définir les relations entre les tables à l’aide des clés étrangères. **(2 pts)**

4. Configurer le fichier **.env** de Laravel pour assurer la connexion à la base de données distante. **(2 pts)**
-

Dossier 2 : Développement de l'application Laravel Web + API (25 pts)

A. Mise en place de l'application Laravel

1. Créer l'application Laravel et la connecter à la base de données AlwaysData. **(2 pts)**
 2. Créer les migrations nécessaires pour générer la structure des tables demandées. **(3 pts)**
 3. Créer les modèles Eloquent correspondants avec les relations appropriées. **(3 pts)**
-

B. Partie Web Laravel avec Blade

4. Créer un contrôleur web pour la gestion des emprunts. **(2 pts)**
5. Créer une vue Blade nommée **listeEmprunts.blade.php** permettant d'afficher les emprunts sous forme de tableau. **(3 pts)**

Le tableau doit afficher au minimum :

- l'identifiant de l'emprunt
 - le titre du livre
 - le nom complet du membre
 - la date d'emprunt
 - la date de retour prévue
 - le statut de l'emprunt
6. Créer une vue Blade nommée **ajouterEmprunt.blade.php** contenant un formulaire d'ajout d'un emprunt. **(3 pts)**
 7. Créer les routes web nécessaires pour :
 - afficher la liste des emprunts
 - afficher le formulaire d'ajout
 - enregistrer un nouvel emprunt **(2 pts)**
-

C. Partie API Laravel

8. Créer un contrôleur API permettant de : **(4 pts)**
 - retourner la liste des livres
 - retourner la liste des emprunts
 - enregistrer un nouvel emprunt
 9. Créer les routes API correspondantes. **(2 pts)**
 10. Configurer CORS pour autoriser les requêtes provenant de l'application React déployée. **(2 pts)**
 11. Vérifier l'accessibilité des routes API. **(2 pts)**
-

Dossier 3 : Gestion d'état avec Redux (Partie indépendante de l'API) (10 pts)

Travail demandé

Dans cette partie, il n'est pas demandé d'effectuer d'appel API.

Le travail doit être réalisé à partir d'un **store Redux local**.

On considère l'état initial suivant :

```
const initialState = {  
  livres: [  
    { id: 1, titre: "L'Alchimiste", isbn: "978-1", nombre_exemplaires: 5, statut: "Disponible" },  
    { id: 2, titre: "Le Petit Prince", isbn: "978-2", nombre_exemplaires: 2, statut: "Disponible" }  
  ],  
  emprunts: [  
    { id: 1, id_livre: 1, id_membre: 1, date_emprunt: "2026-04-10", date_retour_prevue: "2026-04-20", statut: "En cours" }  
  ]  
};
```

Travail demandé

1. Configurer Redux dans l'application React. **(2 pts)**
2. Créer un store Redux contenant au minimum :
 - la liste des livres
 - la liste des emprunts **(2 pts)**
3. Créer les actions Redux nécessaires pour :

- ajouter un emprunt
 - supprimer un emprunt
 - filtrer les livres par statut **(3 pts)**
4. Créer un reducer ou slice Redux permettant de gérer ces actions. **(2 pts)**
 5. Intégrer le store à l'application React via **Provider**. **(1 pt)**
-

Dossier 4 : Développement de l'application React et consommation de l'API (15 pts)

A. Partie interface React utilisant Redux local

1. Créer les composants suivants en exploitant les données du store Redux : **(5 pts)**
 - **ListeLivres**
 - affiche la liste des livres
 - permet de filtrer les livres par statut
 - **ListeEmprunts**
 - affiche la liste des emprunts enregistrés dans le store
 - **AjouterEmpruntLocal**
 - permet d'ajouter un emprunt dans le store Redux sans appel API
-

B. Partie communication avec l'API Laravel

2. Créer un composant **ListeLivresApi** qui récupère la liste des livres depuis l'API Laravel et l'affiche. **(3 pts)**
 3. Créer un composant **ListeEmpruntsApi** qui récupère la liste des emprunts depuis l'API Laravel et l'affiche. **(2 pts)**
 4. Créer un composant **AjouterEmpruntApi** qui envoie un nouvel emprunt vers l'API Laravel à l'aide d'une requête HTTP. **(3 pts)**
 5. Configurer le routage React pour naviguer entre les différentes pages de l'application. **(2 pts)**
-

Dossier 5 : Déploiement en ligne (10 pts)

Travail demandé

1. Déployer l'application Laravel sur **Render** ou **Vercel**. (4 pts)

L'application Laravel déployée doit permettre :

- l'accès aux vues Blade
 - l'accès aux routes API
2. Vérifier l'accessibilité en ligne :
 - d'au moins une vue Blade
 - d'au moins une route API (2 pts)
 3. Déployer l'application React sur **Vercel**. (2 pts)
 4. Configurer l'URL de l'API distante dans l'application React et vérifier la communication entre React et Laravel. (2 pts)

Contraintes techniques

Le candidat doit respecter les contraintes suivantes :

- la base de données doit être hébergée sur **AlwaysData**
 - l'application Laravel doit contenir :
 - une partie **web avec Blade**
 - une partie **API REST**
 - l'application React doit contenir :
 - une partie **Redux locale indépendante de l'API**
 - une partie **communication avec l'API Laravel**
 - les applications doivent être déployées en ligne
 - la communication entre React et Laravel doit être fonctionnelle
-

Livrables attendus

Le candidat doit fournir :

1. le projet **Laravel**
2. le projet **React**
3. les informations de configuration nécessaires
4. les liens de déploiement :
 - lien de l'application Laravel
 - lien d'une vue Blade accessible
 - lien d'une route API accessible
 - lien de l'application React

Barème global

Dossier	Points
Base de données	10 pts
Laravel Web + API	20 pts
Redux	10 pts
React + API	10 pts
Déploiement	10 pts
Qualité projet	10 pts
Présentation orale	10 pts
Total	/80 pts